Listing 16.10 Creating a `Source.queue`

```
class QueueSpec ... {
  ...

  "Source queue" should {
    "allow adding elements to a stream" in {

      val bufferSize = 10                                                    ❶

      val queue: BoundedSourceQueue[Int] =
        Source.queue[Int](                                                   ❷
          bufferSize
        )
        .map(x => 2 * x)
        .toMat(Sink.foreach(x => println(s"processed $x")))(Keep.left)       ❸
        .run()

        (1 to 10).map(x => queue.offer(x))
    }
  }
  ...
}
```

❶ Max elements to hold before they are processed downstream

❷ Creates the BoundedSourceQueue that drops messages when overflowing

❸ Materialization

You need to pass a `bufferSize` to the `Source.queue`. After all, it is a queue, and it has limits. When the buffer is full, elements are dropped, depending on your strategy. You can drop the new elements added to the stream or existing elements from the buffer.

The signature is this:

```
Source.queue[T](bufferSize: Int): Source[T, BoundedSourceQueue[T]]
```

You need to use its materialized value to get the `BoundedSourceQueue`. You may recall that `Source[T, NotUsed]` is a common form of many sources. But here—similar to